

Lecture 2 - Floyd-Hoare Logic and Weakest pre-conditions

In this lecture, we introduce Floyd's assertions to specify programs and triples, and Hoare's logic to prove that programs satisfy a given specification. We then introduce Dijkstra's weakest precondition calculus to guide semi-automatic proof search in the Dafny tool.

Hoare triples

Hoare triples are formulas composed of logical formulas and program statements. Triples are intended to express properties of the state before and after the execution of the program represented by the statement.

Remember that a state is a (partial) function from program variables to values; formally, we set

```
s : State = Var --> Val
```

Let S be a program statement and $s, t: \text{State}$; then define the predicate

```
(S, s) => t if the execution of S starts with s  
normally terminates with t
```

Let P and Q be assertions, namely predicates of the states

```
P, Q: State --> Bool
```

and let S be a program statement. Then we call $\{P\} S \{Q\}$ a "Hoare-triple" and say that it is **valid** if

```
forall s, t : State :: P s && (S, s) => t ==> Q s
```

Notice that if $(S, s) \Rightarrow t$ does not hold, the implication above is vacuously true. This is the "partial correctness" criterion, which is oblivious to proper termination. For this reason, we say that P is a **liberal pre-condition** of Q w.r.t. S .

About termination and the notion of (strong) pre-condition we shall say more later.

The following examples of valid triples can be tested by Dafny:

```
// validity of {x == 0} y := x + 3 {y < 100}
method HoareTriple1(x: int)
{
    assume x == 0; // an assumption always succeeds
    var y := x + 3;
    assert y < 100;
}

// validity of {x == 0} y := x + 3 {0 <= x && y == 3}
method HoareTriple2(x: int)
{
    assume x == 0;
    var y := x + 3;
    assert 0 <= x && y == 3;
}

// validity of {x == 0} y := x + 3 {true}
method HoareTriple3(x: int)
{
    assume x == 0;
    var y := x + 3;
    assert true;
}
```

Hoare logic

Hoare logic is a system of rules for deriving valid triples. In the original formulation, program statements are limited to those generated by the following grammar.

```
S, T ::= skip | x := a | S;T | if b then S else T | while b do S
```

where x is a program variable, a is an arithmetic expression, possibly including program variables, b is a boolean expression such as

```
x == 0, x != y (namely !(x == y))
x > 4 && y = x * 3, 0 <= x + 2 || y > 7
```

etc., including the boolean constants `true` and `false`.

Executions of program statements are understood as in ordinary imperative languages like C, except for the `skip` statement, which terminates, leaving the state unaffected:

```
forall s : State :: (skip, s) => s
```

Since `skip` leaves the state unchanged, the rule for `skip` is just

```
skip -----
{P} skip {P}
```

for all P.

The case of assignment $x := a$ requires defining the meaning of a in a given state s , which is the evaluation of a in s . Suppose that the grammar of arithmetic expressions is

```
a, a' ::= x | c | a + a' | ...
```

where x is any variable, c represents numerical constants 0, 1, ... Then we define $\text{aval } a \text{ } s$ by cases:

```
aval x s = s x
aval c s = c      (identifying the constant
                   symbol c with its value)
aval (a + a') s = (aval a s) + (aval a' s)
...
```

In the third equation, the $+$ on the left-hand side of $=$ is just a symbol, while the $+$ on the right-hand side is the arithmetical sum function.

Let $s[x := n]$ be the state defined by

```
s[x := n] y := if y == x then n else s y
```

and the predicate $P[a/x]$ be defined by

```
P[a/x] s := P (s[x := aval a s])
```

Then we can write the rule:

```
assn -----
{P[a/x]} x := a {P}
```

Exercise: argue that $\{P[a/x]\} x := a \{P\}$ is valid.

The writing $P[a/x]$ is reminiscent of substitution in logic. For this to make sense, note that in Hoare logic and in Dafny, predicates are defined by means of logical formulas. For example:

```
F := y <= 43    which defines the predicate P_F s := (s y <= 43)
```

where $s y = \text{aval } y \text{ } s$ and $43 = \text{aval } 43 \text{ } s$. Now $F[x + 1/y]$ is the result of substituting each occurrence of y by $x + 1$, hence

```
F[x + 1/y] == x + 1 <= 43
```

Therefore, instances of *assn* are

```
-----
{x + 1 <= 43} y := x + 1 {y <= 43}
```

and

$$\frac{}{\{x + 1 \leq 43\} \ x := x + 1 \ \{x \leq 43\}}$$

Example. The following can be checked by Dafny:

```
method HoareTriple4()  
{  
  var x := 100;  
  assume x + 1 <= 43;  
  x := x + 1;  
  assert x <= 43;  
}
```

Now, how to deduce $\{x < 43\} \ x := x + 1 \ \{x \leq 43\}$? For this rule, *assn* does not suffice; indeed, Hoare logic has a rule "*cons*" for consequence:

$$\text{cons} \frac{P \implies P' \quad \{P'\} \ S \ \{Q'\} \quad Q' \implies Q}{\{P\} \ S \ \{Q\}}$$

where $P \implies P'$ is defined as $\text{forall } s :: P \ s \implies P' \ s$ and $P \ s \implies P' \ s$ is a valid implication of arithmetic; similarly for $Q \implies Q'$. When $P = P'$ ($Q' = Q$), namely, are true of the same states, then the first (third) premise can be omitted.

Now the desired proof is

$$\text{cons} \frac{x < 43 \implies x + 1 \leq 43 \quad \text{assn} \frac{}{\{x + 1 \leq 43\} \ x := x + 1 \ \{x \leq 43\}}}{\{x < 43\} \ x := x + 1 \ \{x \leq 43\}}$$

A notation for program-proofs

Instead of using the tree form, the previous proof can be written as a program with predicates interspersed among the statements (here, we have just one):

```
{x < 43}           // 1  
{x + 1 <= 43}     // 2  
x := x + 1;  
{x <= 43}         // 3
```

Predicates 1 and 2 are written on consecutive lines, meaning that $1 \implies 2$. On the other hand, when there is a statement between two predicates, as in 2 and 3, we mean that an instance of rule *assn* has been used.

Example.

```

method swap1()
{
  var x, y := 57, 12;
  // the choices of 52 and 12 are arbitrary: try others
  ghost var X, Y := x, y;
  // ghost variables are not part of the code

  assert x == X && y == Y;
  assert y == Y && x == X;
  // by (x == X && y == Y) ==> (y == Y && x == X)
  var tmp := x;
  assert y == Y && tmp == X;
  x := y;
  assert x == Y && tmp == X;
  y := tmp;
  assert x == Y && y == X;
}

```

Composition rule

Let $S ; S'$ be the sequential composition of S and S' , whose execution consists of first executing S and, if and when this phase ends, continuing executing S' from the state reached in the execution of S .

The rule for the sequence is:

$$\text{seq} \frac{\{P\} S \{Q\} \quad \{Q\} S' \{R\}}{\{P\} S ; S' \{R\}}$$

To argue that this rule is sound, let s be such that $P s$ and $(S ; S') \Rightarrow r$. Then there must exist t such that $(S, s) \Rightarrow t$ and $(S', t) \Rightarrow r$. Then, by the validity of $\{P\} S \{Q\}$, it follows that $Q t$; by the validity of $\{Q\} S' \{R\}$, it follows that $R r$, as desired.

This rule has been implicitly used in example `swap1()`. The triples

- (a) $\{y == Y \ \&\& \ x == X\} \text{ tmp} := x \ \{y == Y \ \&\& \ \text{tmp} == X\}$
- (b) $\{y == Y \ \&\& \ \text{tmp} == X\} x := y \ \{x == Y \ \&\& \ \text{tmp} == X\}$
- (c) $\{x == Y \ \&\& \ \text{tmp} == X\} y := \text{tmp} \ \{x == Y \ \&\& \ y == X\}$

are instances of the rule (asn); then

- (d) $\{y == Y \ \&\& \ x == X\}$
 $\text{tmp} := x ; x := y$
 $\{x == Y \ \&\& \ \text{tmp} == X\}$

follows from (a) and (b) by (seq); finally

```
(e) {y == Y && x == X}
    tmp := x ; x := y ; y := tmp
    {x == Y && y == X}
```

follows from (e) and (c) again by (seq).

The same proof, but written as a program with assertions, is example swap1().

Exercise: Fill the missing assertions in the proof that Dafny performs automatically

```
method swap2()
{
  var x, y := 57, 12;
  // the choices of 52 and 12 are arbitrary
  ghost var X, Y := x, y;

  assert x == X && y == Y;
  x := y - x;
  y := y - x;
  x := x + y;
  assert x == Y && y == X;
}
```

Conditional rule

```

{P && b} S {Q}      {P && !b} T {Q}
if -----
    {P} if b then S else T {Q}
```

Exercise: state and derive suitable pre- and post-conditions for the statement

```
if x < y then m := y else m := x
```

Then write a Dafny method to check your answers.

```
method Max(x: int, y: int) returns (m: int)
  ensures m == x || m == y
  ensures x <= m && y <= m
{
  if x < y {
    m := y;
    assert m == y >= x;
  }
  else {
    assert y <= x;
    m := x;
    assert m == x >= y;
  }
}
```

Exercise: A rule is "derivable" w.r.t. a formal system if it can be viewed as abbreviating a derivation whose leaves are labeled by the rule's premises and whose root is labeled by its conclusion.

Prove that the following rule is derivable:

$$\frac{P \ \&\& \ b \implies U \quad \{U\} \ S \ \{X\} \quad P \ \&\& \ !b \implies V \quad \{V\} \ T \ \{Y\}}{\{P\} \ \text{if } b \text{ then } S \ \text{else } T \ \{X \ || \ Y\}}$$

Weakest preconditions

We assume that the programming language we are considering is "deterministic," meaning that for all program statements S and states s , if $(S, s) \Rightarrow t$ for some state t , then t is unique.

Definition. Let P and Q be state predicates and S a program statement; we say that P is a (strong) **pre-condition** of Q w.r.t S if

$$\text{forall } s :: P \ s \implies \text{exists } t :: (S, s) \Rightarrow t \ \&\& \ Q \ t$$

Define the predicate $\text{wp}(S, Q)$ as the **weakest (strong) precondition** namely

i) $\text{wp}(S, Q)$ is a strong pre-condition of Q w.r.t. S and

ii) for all strong pre-condition P of Q w.r.t. S it holds $P \implies \text{wp}(S, Q)$

where $P \implies Q$ if $\text{forall } s :: P \ s \implies Q \ s$.

Then an explicit definition of $\text{wp}(S, Q)$ is:

$$\text{wp}(S, Q) \ s \iff \text{exists } t :: (S, s) \Rightarrow t \ \&\& \ Q \ t$$

Total preconditions are a subset of partial preconditions, but not vice versa. The difference becomes apparent when considering programs that may not terminate; however, if we discard loops and recursive procedures (which we shall treat later), then all programs terminate in any state.

Proposition. (total versus partial pre-conditions)

1. If P is a total pre-condition of Q w.r.t. S , then the triple $\{P\} \ S \ \{T\}$ is valid
2. if $\{P\} \ S \ \{T\}$ is valid and S terminates from any s s.t. $P \ s$, then P is a total pre-condition of Q w.r.t. S .

For any program statement S , the mapping $\text{wp}(S, _)$ (where $\text{wp}(S, _) \ Q = \text{wp}(S, Q)$) is a function from predicates to predicates, called a "predicate transformer". The interest of $\text{wp}(S, _)$ is that, at least when S does not include loops or procedure calls, it can be given an explicit definition:

Proposition. (wp-properties)

1. $wp(x := a, Q) = Q[a/x]$
2. $wp(S;T, Q) = wp(S, wp(T, Q))$
3. $wp(\text{if } b \text{ then } S \text{ else } T, Q) = (b' \ \&\&' \ wp(S, Q)) \ \text{II}' \ (!b' \ \&\&' \ wp(T, Q))$

where $b' \ s = \text{bval } b \ s$ (bval evaluates the boolean expression b in the state s), $!b' \ s = \neg(\text{bval } b \ s)$, $(P \ \&\&' \ Q) \ s = P \ s \ \&\& \ Q \ s$, $(P \ \text{II}' \ Q) \ s = P \ s \ \text{II} \ Q \ s$.

Theorem. (wp-Lemma) If S does not contain loops or procedure calls, then for all Q :

1. $\{wp(S, Q)\} S \{Q\}$ is derivable in Hoare logic, hence valid
2. $\{P\} S \{Q\}$ is derivable if and only if $P \implies wp(S, Q)$.

As a consequence of the wp-Lemma, we can compute the weakest precondition of a program S by reasoning backward, starting with a postcondition Q . To verify S against the specification with precondition P , it suffices to prove $P \implies wp(S, Q)$ using only mathematical logic and arithmetic.

Exercise

```
method MyMethod(x: int) returns (y: int)
  requires 10 <= x
  ensures 25 <= y
{
  // 1
  var a: int, b: int;
  // 2
  a := x + 3;
  // 3
  if x < 20 {
    // 4
    b := 32 - x;
    // 5
  } else {
    // (6)
    b := 16;
    // (7)
  }
  // (8)
  y := a + b;
  // (9)
}
```

Assuming that the requires clause (the pre-condition) and the ensure clause (the post-condition) have been copied to positions (1) and (9) respectively, compute assertions from (9) to (1) by means of wp, starting with

```
(9) := 25 <= y    (the post-condition)
(8) := wp(y := a + b, (9))
...
(1) := wp(a := x + 3, (2))
```


Then check your answers using Dafny.

Solution

```
method MyMethod(x: int) returns (y: int)
  requires 10 <= x
  ensures 25 <= y
{
  assert 10 <= x; // (1)
  var a: int, b: int;
  assert (x < 20 && 25 <= x + 3 + 32 - x) ||
    (!(x < 20) && 25 <= x + 3 + 16); // (2)
  a := x + 3;
  assert (x < 20 && 25 <= a + 32 - x) ||
    (!(x < 20) && 25 <= a + 16); // (3)
  if x < 20 {
    assert 25 <= a + 32 - x; // (4)
    b := 32 - x;
    assert 25 <= a + b; // (5)
  } else {
    assert 25 <= a + 16; // (6)
    b := 16;
    assert 25 <= a + b; // (7)
  }
  assert 25 <= a + b; // (8)
  y := a + b;
  assert 25 <= y; // (9)
```

Reference: see Leino, chapter 2.